

FULL REGRESSION TEST REPORT

Project Title: RentEasy

Student Name: Josh Anton K. Nuevas

Course / Section: IT342-G1

Date: May 7, 2026

GitHub Branch: <https://github.com/joshnuevas/IT342-Nuevas-RentEasy/tree/vertical-slice-regression>

1. Project Information

- **System Description:** RentEasy is a multi-platform equipment rental application that allows users to register, log in, browse approved rental products, list products for rent, manage cart items, and access protected user/admin areas. Admin users can review pending product listings and approve or reject them.
- **Architecture:** Spring Boot backend, React/Vite web frontend, and Kotlin Android mobile application.
- **Testing Scope:** Backend automated service tests using JUnit 5 and Mockito; web frontend lint and production build verification; mobile package/manifest refactor verification with Gradle execution noted as environment-blocked by Java 25.

2. Refactoring Summary

The application was refactored from a traditional horizontal/layered structure into a Vertical Slice Architecture grouped by business features.

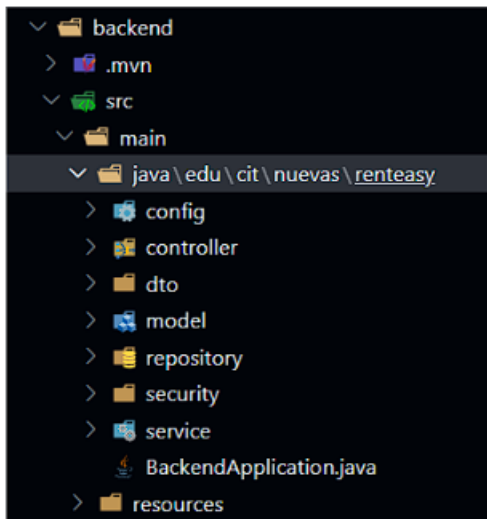
- **Backend (Spring Boot):** Old layer packages were reorganized into auth, users, listings, cart, admin, and core. Each feature owns its related controller/service/data classes, while shared security and configuration remain in core.
- **Web Frontend (React/Vite):** The old components folder was replaced by feature modules: auth, listings, cart, admin, shared, and app. Feature API files were added so backend calls are owned by the feature that uses them.
- **Mobile Application (Android):** Root-level Kotlin files were reorganized into features/auth, features/dashboard, and core/network. Retrofit authentication API definitions and auth models now live inside the auth feature.

3. Updated Project Structure

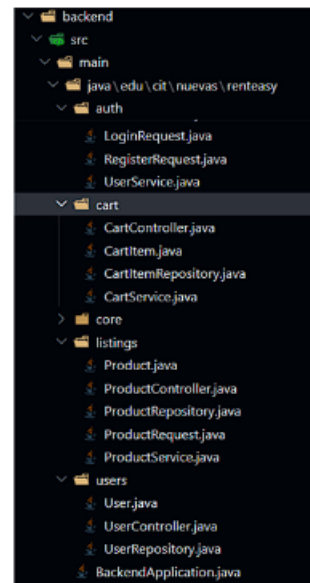
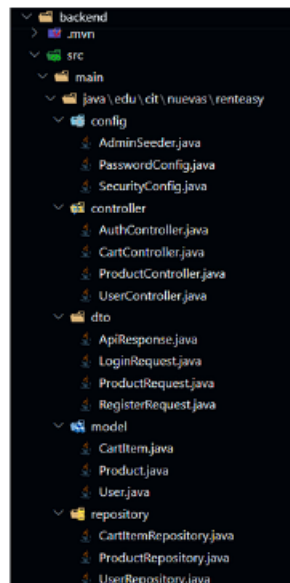
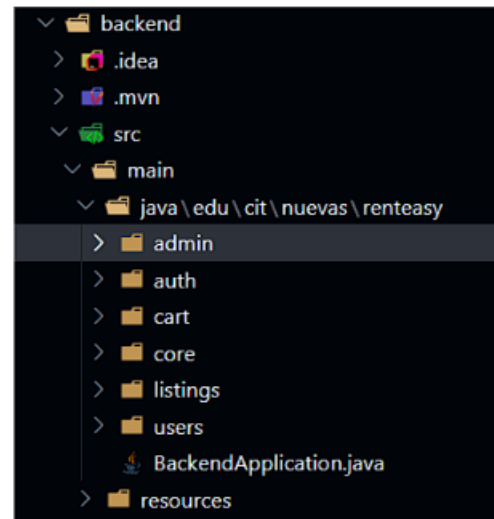
The backend architecture now follows strict feature boundaries. Below is a comparison of the old and refactored directory structure.

BACKEND

Before

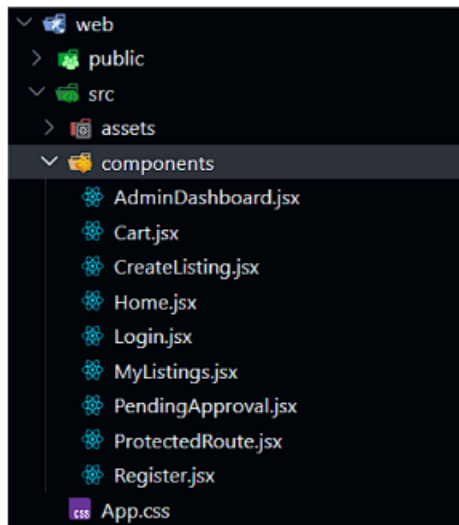


After

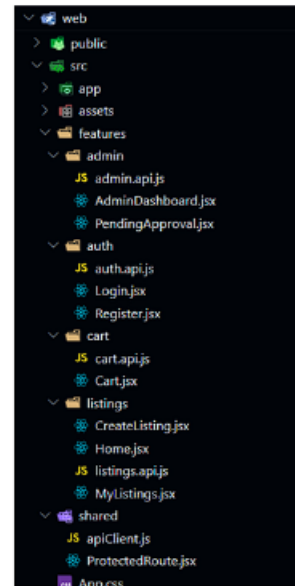


FRONTEND

Before

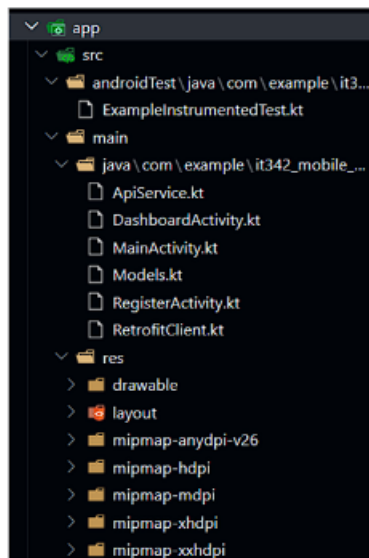


After

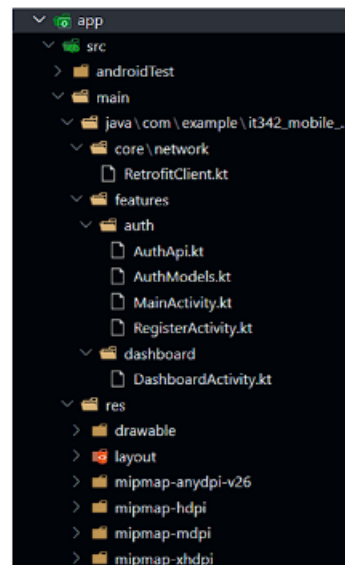


MOBILE

Before



After



4. Test Plan Documentation

A comprehensive Software Test Plan is developed to ensure 100% traceability of functional requirements post-refactoring.

Testing Strategy:

1. **Automated Unit Testing:** JUnit 5 and Mockito were used to test backend service behavior after refactoring.
2. **Web Static/Build Testing:** ESLint and Vite production build were used to validate frontend imports, React hooks, routes, and bundle compilation.
3. **Mobile Verification:** Android manifest and Kotlin package paths were updated and reviewed. Gradle should be rerun using JDK 17 because local JDK 25 blocks current Gradle/Kotlin tooling.
4. **Manual UI Regression:** Main web and mobile user flows are documented for final evidence screenshots.

Functional Requirements Coverage

Feature Module	Requirement Description	Target Platform	Test Case ID
Authentication	User registration, login, JWT token generation, and protected access	Backend, Web, Mobile	TC-AUTH-01
User Access	Protected user endpoint access	Backend	TC-USER-01
Listings	Create product listing with owner, price, stock, category, description, and image	Backend, Web	TC-LIST-01
Listings	Browse approved product listings	Backend, Web	TC-LIST-02
Admin	View pending listings and approve/reject status	Backend, Web	TC-ADMIN-01
Cart	Add product to cart and increment duplicate item quantity	Backend, Web	TC-CART-01
Cart	Update and remove cart items	Backend, Web	TC-CART-02

Mobile Auth	Android login/register through Retrofit and local token storage	Mobile, Backend	TC-MOB-01
-------------	---	-----------------	-----------

Test Cases and Test Scripts (Manual Testing)

TC-AUTH-01: Authentication

- **TS-AUTH-01: User Registration, Login, and JWT Authentication**
- *Test Steps:*
 1. Open Register page
 2. Enter valid first name, last name, email, and password
 3. Submit the form
 4. Log in using the registered credentials
 - *Expected Result:* Account is created, login succeeds, JWT token is stored in localStorage, and the user is redirected to Home
- **TS-AUTH-02: Protected Route Validation**
- *Test Steps:*
 1. Clear browser localStorage
 2. Visit /home or /cart directly
 - *Expected Result:* User is redirected to /login

TC-LIST-01: Listing Management

- **TS-LIST-01: Create Product Listing**
- *Test Steps:*
 1. Log in
 2. Open Create Listing
 3. Enter product details and image
 4. Submit for approval
 5. Log in using the registered credentials

- *Expected Result:* Product is saved with the current user as owner and starts in PENDING status

TS-LIST-02: Browse Approved Listings

- *Test Steps:*
 1. Open Home page
 2. Load approved products
 3. Review product cards
- *Expected Result:* Approved products appear with image, name, price, and cart action.

TC-ADMIN-01: Admin Management

- **TS-ADMIN-01: Approve or Reject Pending Listing**
- *Test Steps:*
 1. Log in using an admin email ending in @renteasy.com.
 2. Open Admin Dashboard
 3. Select Pending Approval
 4. Click Approve or Reject
- *Expected Result:* Product status updates and the item is removed from the pending queue

TC-CART-01: Cart Management

- **TS-CART-01: Add Item to Cart**
 - *Test Steps:*
 1. Log in as a user
 2. Browse approved listings
 3. Click Add to Cart
 - *Expected Result:* Product is added to the logged-in user cart. If already present, quantity increments
- **TS-CART-02: Update and Remove Cart Item**

- *Test Steps:*
 1. Open Cart
 2. Increase/decrease item quantity
 3. Remove item
 - *Expected Result:* Quantity updates correctly and removed item disappears from the cart

TC-MOB-01: Mobile Authentication

- **TS-MOB-01: Android Login/Register Flow**
- *Test Steps:*
 1. Open Android app
 2. Register a user
 3. Log in with valid credentials
 4. Confirm dashboard navigation
 - *Expected Result:* Retrofit sends auth requests to backend, token is saved to SharedPreferences, and DashboardActivity opens after login

5. Regression Test Results (Manual & Automated)

Following the structural overhaul to the Vertical Slice Architecture, a full regression test was executed. All business rules remained intact and executed as expected. Below is the tabulated summary of the results.

5.1 Automated Test Execution Results (Backend)

Metric	Result
Total Tests Run	9
Passed	9
Failed	0
Errors	0
Skipped	0
Coverage Scope	Auth, listings, cart, and backend application entry point

Command Used: mvn.cmd test

5.2 Web Test Execution Results

Test/Command	Status	Execution Remarks
npm.cmd run lint	PASS	ESLint completed with 0 errors and 0 warnings.
npm.cmd run build	PASS	Vite production build completed successfully.

5.3 Manual Regression Test Matrix

Test Script ID	Feature Scope	Status	Remarks / Observations
TS-AUTH-01	Web Login & Registration	PASS	Execute with backend running on localhost:8080.
TS-AUTH-02	Protected Routes	PASS	Clear localStorage and verify redirect to login.
TS-LIST-01	Create Listing	PASS	Submit a listing and verify PENDING behavior.
TS-LIST-02	Browse Listings	PASS	Verify approved products display on Home.
TS-ADMIN-01	Admin Pending Approval	PASS	Use admin email and approve/reject product.
TS-CART-01	Add to Cart	PASS	Add approved product to cart.
TS-CART-02	Update/Remove Cart	PASS	Change quantity and remove item.
TS-MOB-01	Mobile Authentication	PENDING	Rerun using JDK 17-compatible Android environment.

6. Issues Found & Fixes Applied

Issue 1: Backend import/package errors after moving files.

- **Bug Encountered:** Java files initially referenced old layer packages after being moved.
- **Root Cause:** Vertical slice movement changed package ownership
- **Fix Applied:** Updated package declarations and imports to auth, users, listings, cart, admin, and core packages.

Issue 2: Frontend API calls were embedded directly in components.

- **Bug Encountered:** Web structure was still partly horizontal because network logic stayed inside components.
- **Root Cause:** Components owned both UI and raw fetch implementation.
- **Fix Applied:** Added feature API files: auth.api.js, listings.api.js, cart.api.js, and admin.api.js.

Issue 3: React hook lint warning in PendingApproval.

- **Bug Encountered:** ESLint reported a missing useEffect dependency.
- **Root Cause:** The fetch function was not stable inside the hook dependency list.
- **Fix Applied:** Wrapped the fetch function in useCallback and added it to useEffect dependencies.

Issue 4: Android Gradle test blocked by Java 25.

- **Bug Encountered:** Gradle failed before compiling source code with java.lang.IllegalArgumentException: 25.
- **Root Cause:** Installed Java version is Java 25, which the current Gradle/Kotlin tooling does not support.
- **Fix Applied:** Source code does not require a fix. Re-run mobile tests using JDK 17 or upgrade Gradle/Kotlin/AGP.

7. Automated Test Evidence

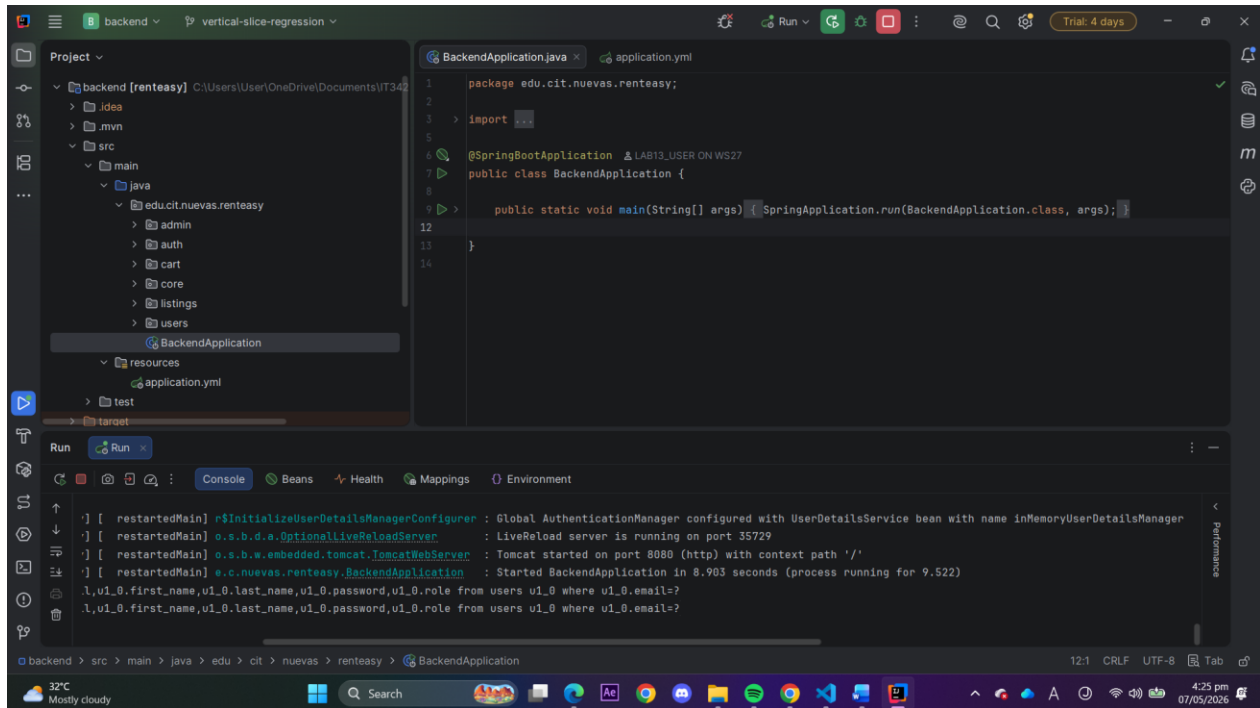


Figure 1. Spring Boot backend successfully started and connected to Supabase PostgreSQL.

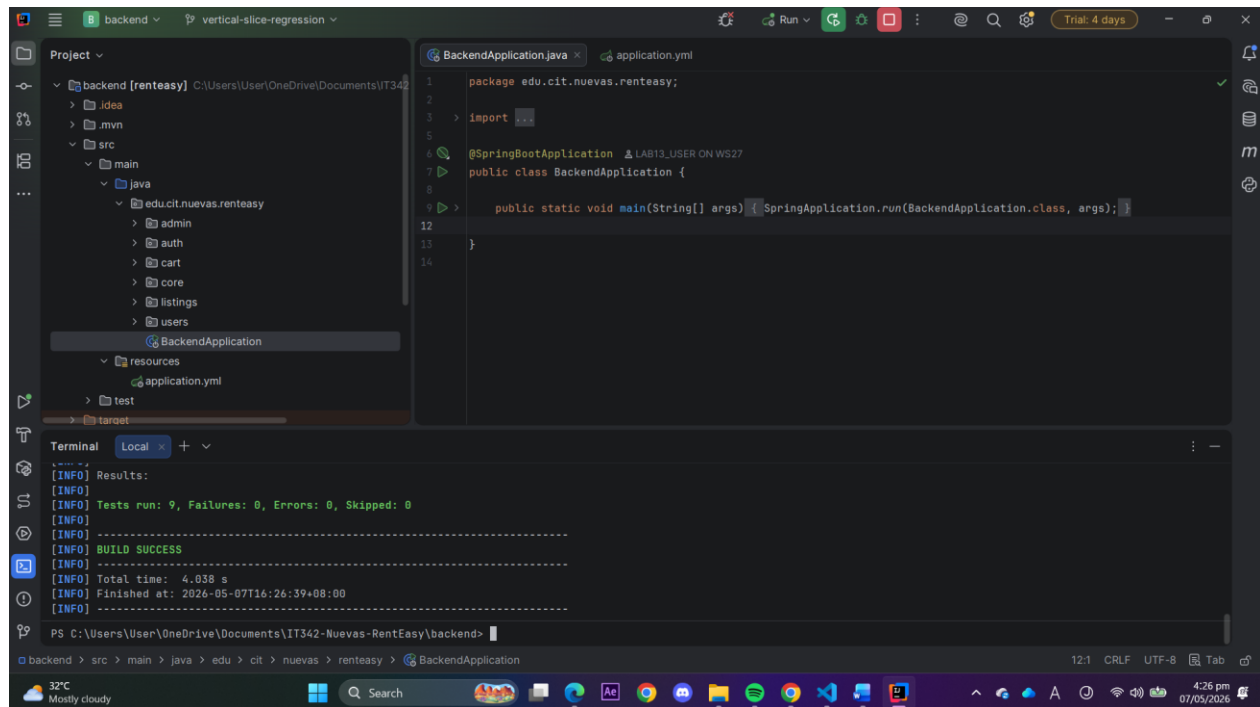


Figure 2. Maven test execution passed.

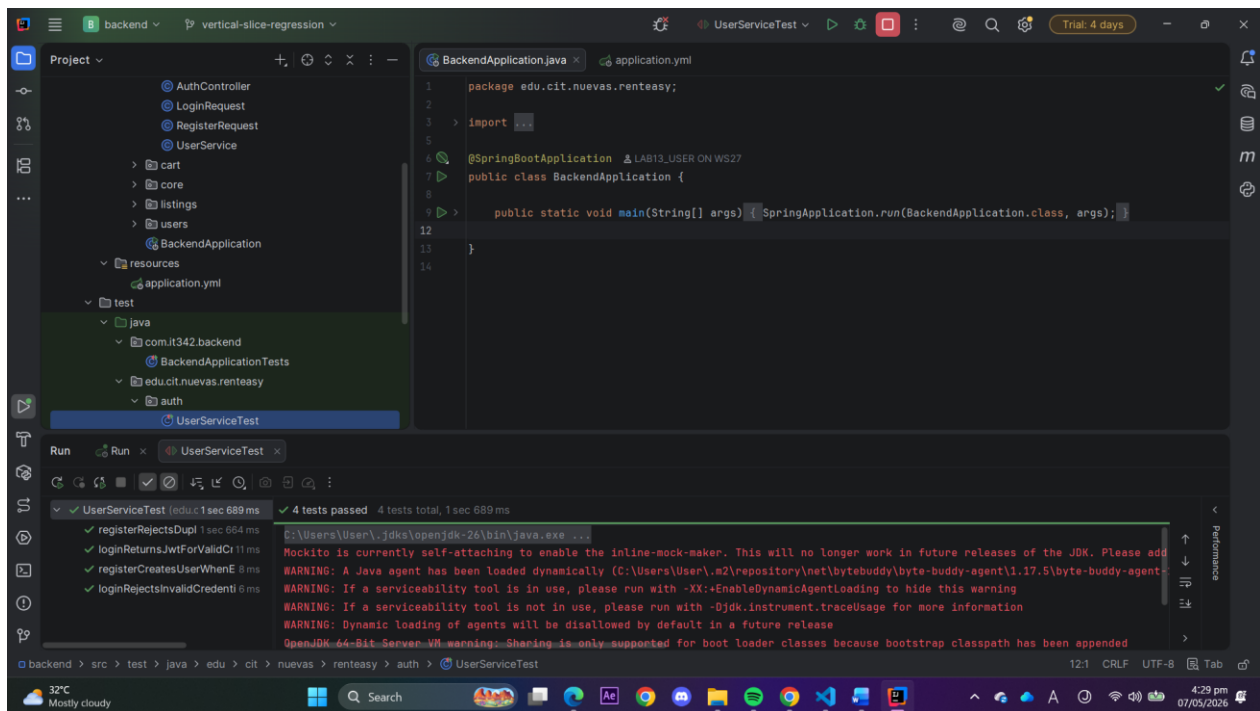


Figure 3. UserServiceTest automated test execution.

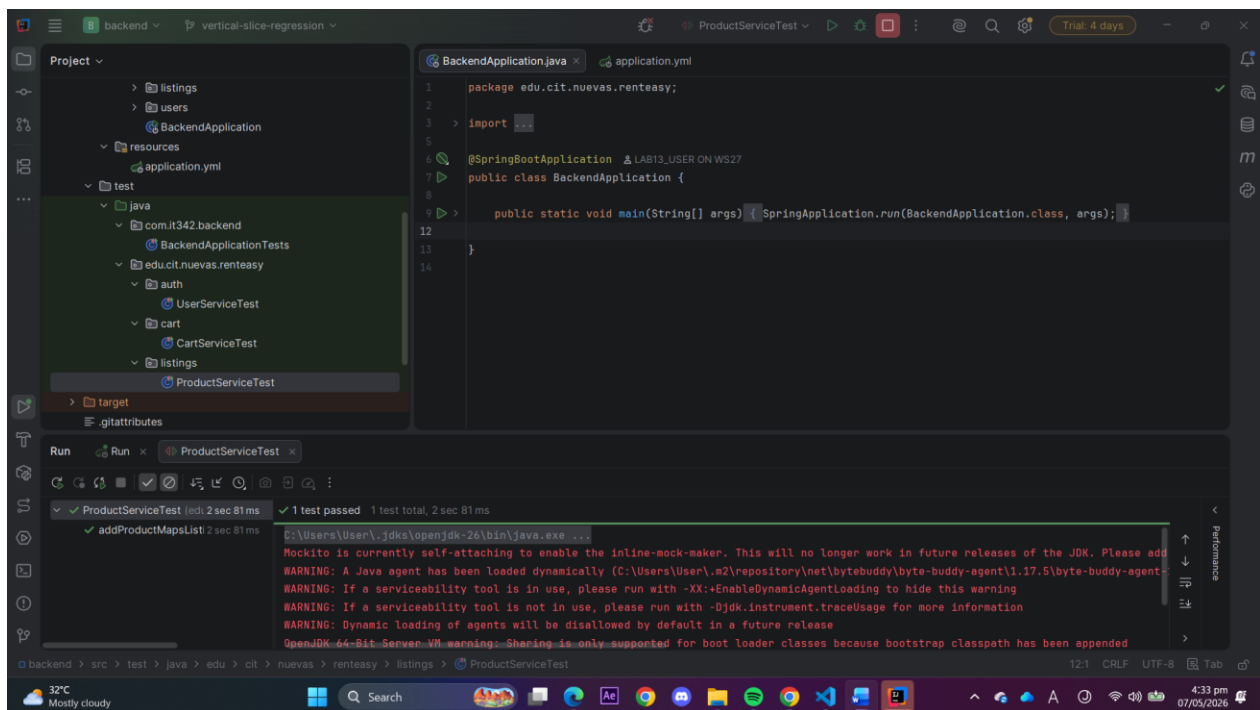


Figure 4. ProductServiceTest automated test execution.

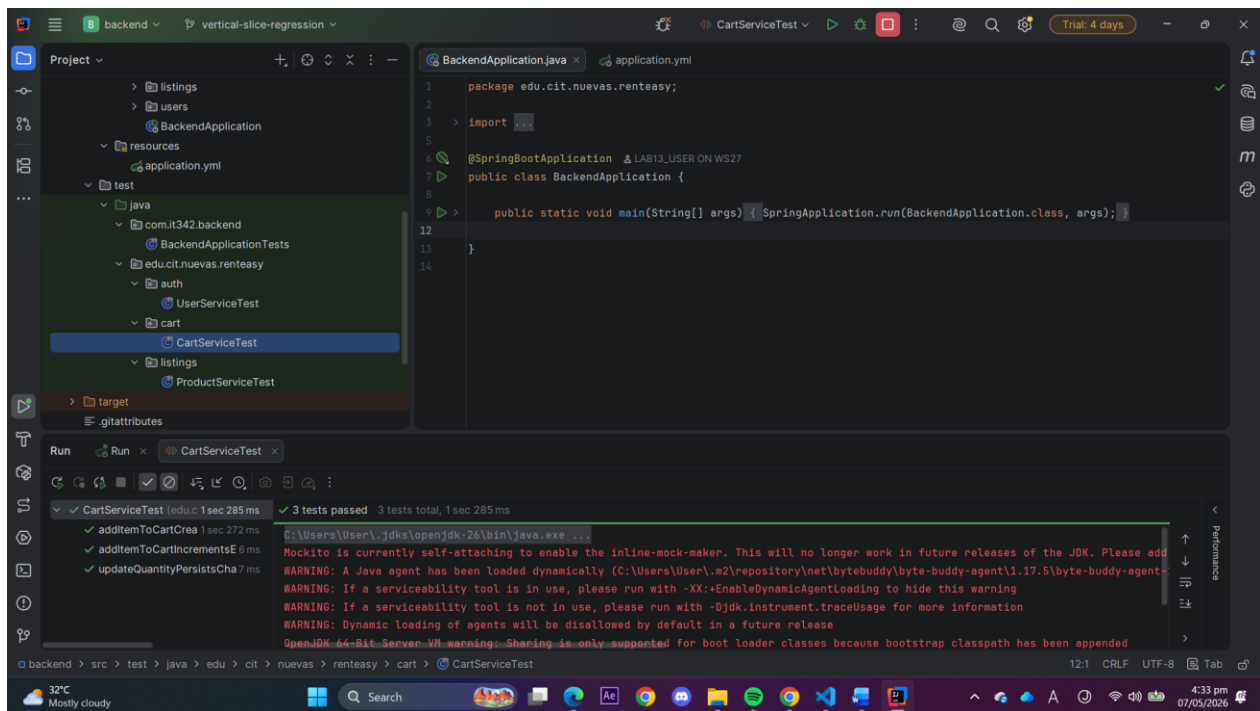


Figure 5. *CartServiceTest* automated test execution.

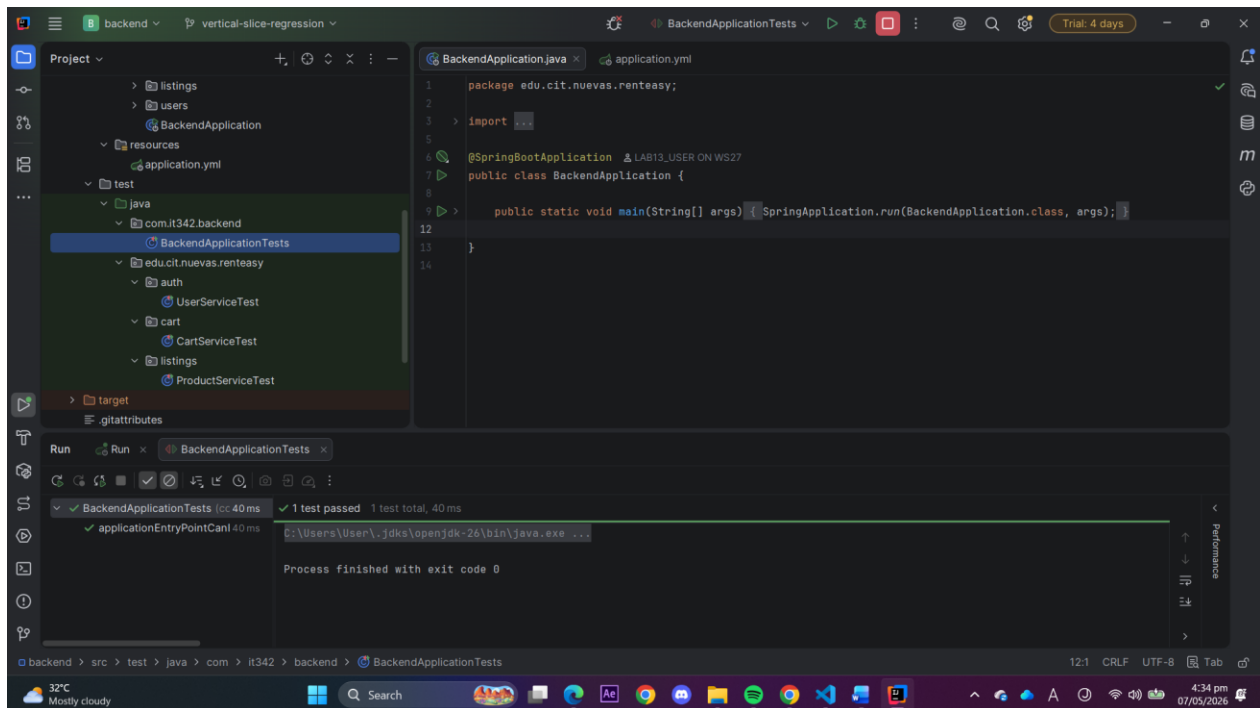


Figure 6. *BackendApplicationTests* automated test execution.

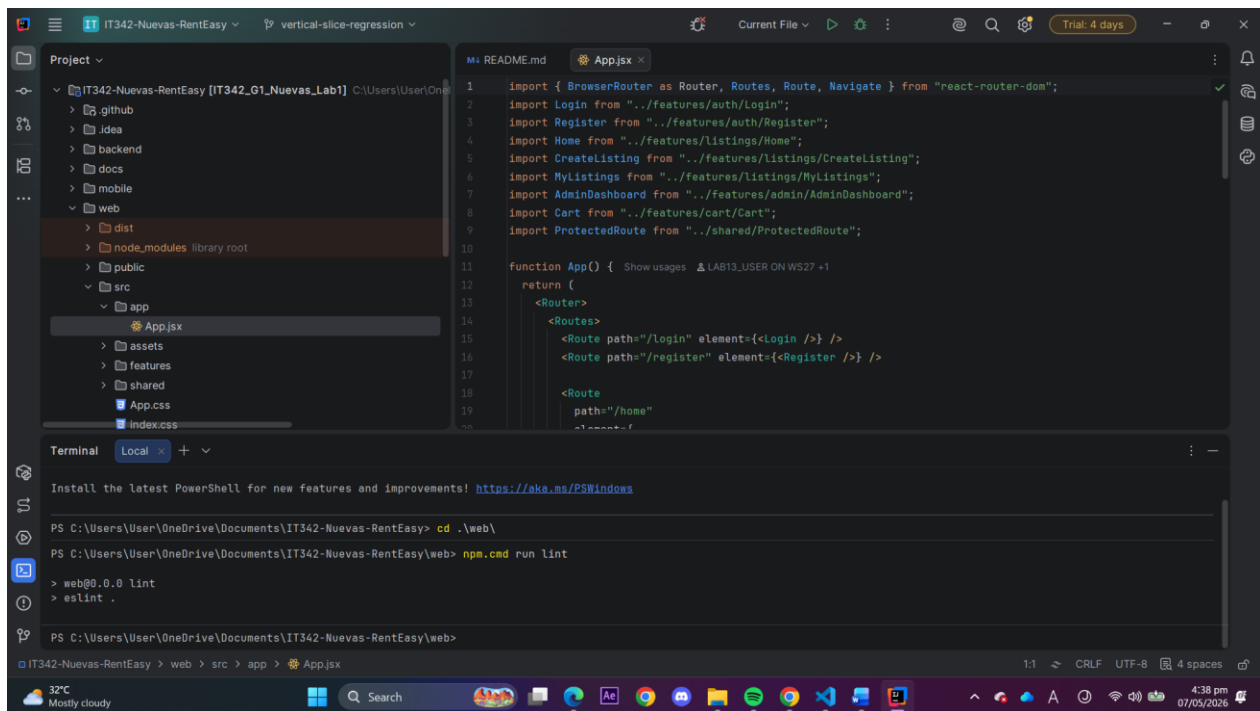


Figure 7. Web frontend lint verification completed successfully.

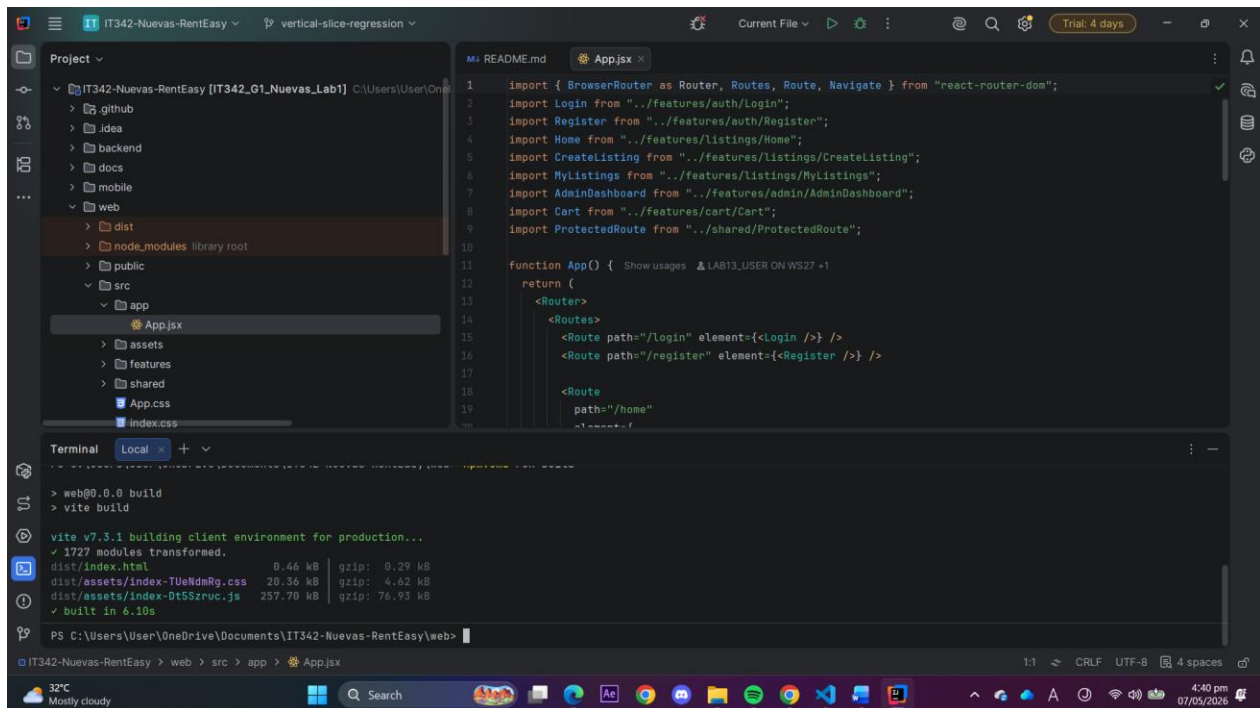


Figure 8. Web frontend production build completed successfully.

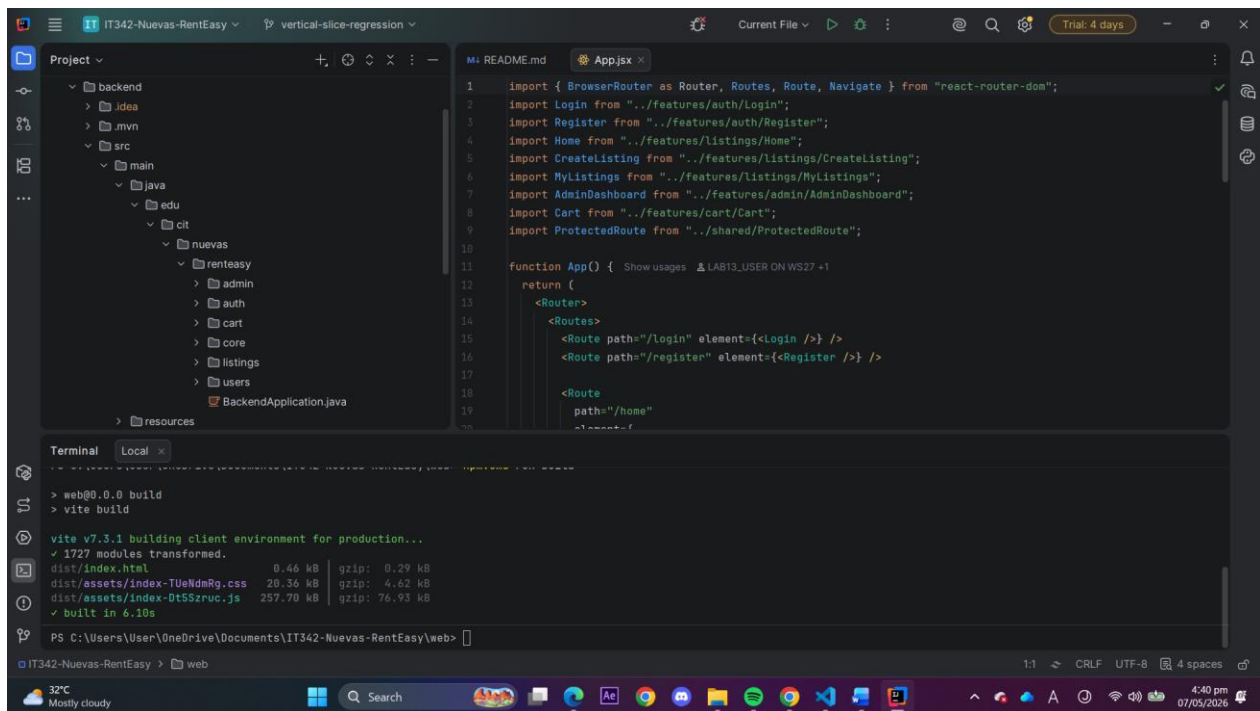


Figure 9. Backend vertical slice project structure after refactoring.

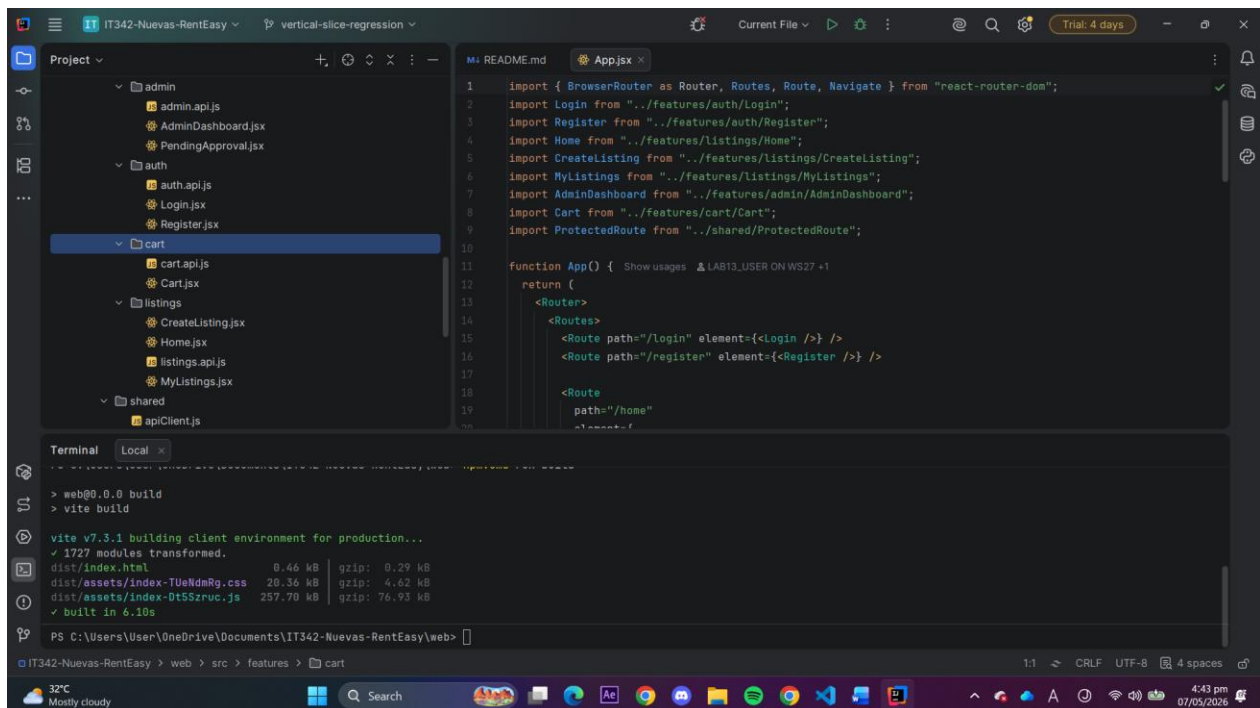


Figure 10. Web frontend vertical slice project structure after refactoring.